

DoomOS Engineering Playbook

Step-by-Step Operating System Build & Implementation Guide

Document: OS Construction & Deployment Manual

Target OS: DoomOS (KDE Plasma 6 / Wayland / Btrfs)

Classification: Technical Architecture & Systems Engineering

Version: 1.0.0

Executive Overview of the Build Pipeline

Building a production-grade Linux operating system follows a deterministic, 7-phase build lifecycle. This playbook details every step required to assemble the root filesystem, inject problem-solving subsystems, brand the desktop environment, configure the installer, and master the final bootable ISO.

DOOMOS 7-PHASE BUILD LIFECYCLE	
Phase 1: Environment & Toolchain	-> Containerized Linux build engine
Phase 2: Base System Bootstrap	-> Kernel (Zen/LTS), firmware, systemd, base packages
Phase 3: Graphics & Desktop	-> SDDM, Wayland, Mesa/NVIDIA, KDE Plasma 6
Phase 4: Custom Subsystems	-> Btrfs/Snapper, DKMS Guardian, PipeWire loopback
Phase 5: Calamares Installer	-> Automated Btrfs partitioning, branding, RTC sync
Phase 6: ISO Mastering	-> Squashfs compression, GRUB hybrid UEFI/BIOS boot
Phase 7: Automated QA & Test	-> QEMU virtual test suite & bare-metal deployment

Phase 1: Build Environment & Directory Scaffolding

Because the host development machine is macOS (Apple Silicon), the build process executes within an isolated, privileged container to ensure a native, case-sensitive Linux build filesystem.

Step 1.1: Project Directory Structure Setup

The build workspace is organized into discrete modular layers:

```

doomos-builder/
├── Dockerfile           # Build engine container definition
├── build.sh             # Master one-click compilation script
├── profile/            # OS definition profiles
│   ├── packages.x86_64 # Manifest of all packages included in ISO
│   ├── pacman.conf     # Package repositories and mirrors
│   └── airootfs/        # File overlay applied to root filesystem
│       ├── etc/
│       │   ├── os-release # Distro identity and metadata
│       │   ├── sysctl.d/  # Kernel performance & inotify tuning
│       │   ├── systemd/sleep.conf.d/ # Laptop battery/suspend optimization
│       │   └── xdg/       # Default KDE Plasma 6 themes and panels
│       └── usr/
│           ├── bin/doom-game # Gaming HDR / Gamescope launch wrapper
│           └── share/
│               ├── calamares/ # Installer branding and configurations
│               └── sddm/themes/ # Login screen theme
└── output/              # Generated bootable ISO files and checksums

```

Step 1.2: Containerized Build Engine Provisioning

The builder runs via a dedicated Linux container with `CAP_SYS_ADMIN` privileges (required for `mount`, `chroot`, and `squashfs` operations):

```

# Dockerfile
FROM archlinux:base-devel
RUN pacman -Syu --noconfirm archiso git squashfs-tools dosfstools libisoburn xorriso
WORKDIR /build
ENTRYPOINT ["/build/build.sh"]

```

Phase 2: Base System Bootstrapping (Rootfs Creation)

Step 2.1: Bootstrapping the Minimal Root Filesystem

Using the package bootstrap engine (`pacstrap`), a clean root filesystem (`rootfs`) is initialized with core hardware and userspace packages:

1. Kernel Tier:

- `linux-zen` (low-latency scheduling, split-lock mitigation, optimized desktop throughput)
- `linux-lts` (long-term stability fallback kernel)
- `linux-firmware` (complete hardware firmware database for Intel, AMD, Realtek, Broadcom)

2. Core System Infrastructure:

- `systemd`, `systemd-sysvcompat`, `dbus`, `polkit`, `sudo`
- `btrfs-progs`, `e2fsprogs`, `dosfstools`, `zram-generator`
- `networkmanager`, `iw`, `bluez`, `bluez-utils`

Step 2.2: Initramfs Generation Architecture

Configure `/etc/mkinitcpio.conf` inside the rootfs:

- **Hooks enabled:** `base udev plymouth autodetect modconf kms keyboard keymap consolefont block btrfs filesystems fsck`
- Generates `vmlinux-linux-zen` and `initramfs-linux-zen.img` supporting early Plymouth boot animation and Btrfs subvolumes.

Phase 3: Graphics, Display Stack & KDE Plasma 6

Step 3.1: Wayland Compositor & Driver Stack

Install the full hardware acceleration and Wayland subsystem:

- **Graphics Drivers:** `mesa`, `vulkan-radeon`, `vulkan-intel`, `nvidia-dkms`, `nvidia-utils`
- **32-bit Compatibility Suite:** `lib32-mesa`, `lib32-vulkan-radeon`, `lib32-vulkan-intel`, `lib32-nvidia-utils`
- **Compositor Core:** `kwin`, `wayland`, `xwayland`, `qt6-wayland`

Step 3.2: Audio & Video Subsystems

- **Audio Stack:** `pipewire`, `wireplumber`, `pipewire-pulse`, `pipewire-alsa`, `pipewire-jack`
- **Multimedia Codecs:** `ffmpeg`, `gststreamer`, `gst-plugins-good`, `gst-plugins-bad`, `gst-plugins-ugly`, `gst-libav`

Step 3.3: Display Manager (SDDM) & Live Autologin

1. Enable SDDM service: `systemctl enable sddm.service`
2. Configure live session autologin in `/etc/sddm.conf.d/liveuser.conf`:

```
[Autologin]
User=liveuser
Session=plasmawayland
```

Phase 4: Implementing the Problem-Solving Subsystems

Step 4.1: Btrfs Automated Snapshot Rollback Setup

1. Install `snapper`, `snapper-pac`, and `grub-btrfs`.
2. Configure `/etc/snapper/configs/root`:
 - `TIMELINE_CREATE=yes`, `NUMBER_LIMIT=10`
3. Configure `snapper-pac` to take pre- and post-transaction snapshots on every update.

4. Enable `grub-btrfsd.service` to dynamically regenerate the GRUB boot menu whenever a snapshot is taken.

Step 4.2: NVIDIA DKMS Guardian Hook

Deploy `/usr/share/libalpm/hooks/90-doomos-nvidia-guardian.hook` :

```
[Trigger]
Operation = Upgrade
Operation = Install
Type = Package
Target = nvidia-dkms
Target = linux-zen

[Action]
Description = DoomOS Guardian: Verifying NVIDIA module compilation...
When = PostTransaction
Exec = /usr/bin/doomos-nvidia-verify
```

The verification script checks module integrity; if missing or broken, it issues an immediate warning and creates a recovery boot fallback.

Step 4.3: Developer Kernel & Inotify Configuration

Deploy `/etc/sysctl.d/99-developer-performance.conf` :

```
# Prevent inotify exhaustion in large codebases
fs.inotify.max_user_watches = 1048576
fs.inotify.max_user_instances = 2048

# Virtual memory mapping limits for databases / JVM
vm.max_map_count = 2147483642

# Network connection backlog
net.core.somaxconn = 8192
```

Step 4.4: Wayland Desktop Portal & Loopback Audio Setup

1. Install `xdg-desktop-portal-kde` and `xdg-desktop-portal-gtk` .
2. Inject virtual audio loopback profile in `/etc/pipewire/pipewire.conf.d/10-loopback.conf` to enable simultaneous microphone and system sound streaming.

Step 4.5: Power & Sleep Optimization

1. Install `auto-cpufreq` and `power-profiles-daemon` .
 2. Enable `auto-cpufreq.service` to intelligently scale CPU states between AC and battery.
 3. Configure `/etc/systemd/sleep.conf.d/10-doomos-sleep.conf` with `MemorySleepMode=deep s2idle` .
-

Phase 5: Calamares Graphical Installer Integration

Step 5.1: Installer Layout & Modules

Configure `/etc/calamares/settings.conf` :

- **Sequence:**
 1. `welcome` (language selection, network status)
 2. `locale` (timezone selection with automatic IP geolocation)
 3. `keyboard` (layout preview)
 4. `partition` (custom DoomOS automated Btrfs partitioning module)
 5. `users` (creation of user, host name, root password)
 6. `unpackfs` (unpacking the squashfs image to target SSD/NVMe)
 7. `doomos-rtc` (Windows dual-boot detection & RTC adjustment)
 8. `bootloader` (GRUB 2 EFI + snapshot menu generation)

Step 5.2: Dual-Boot Windows RTC Sync Script

Script injected into `/usr/lib/calamares/modules/doomos-rtc/main.py` :

```
# Checks if Windows EFI loader exists on any mounted disk
import os, subprocess

if os.path.exists("/boot/efi/EFI/Microsoft/Boot/bootmgfw.efi"):
    subprocess.run(["timedatectl", "set-local-rtc", "1", "--adjust-system-clock"])
```

Phase 6: Squashfs Compression & ISO Mastering

Step 6.1: Compacting Rootfs into Squashfs

The entire root filesystem is compressed using the high-performance Zstandard algorithm:

```
mksquashfs airootfs/ output/airootfs.sfs -comp zstd -Xcompression-level 19 -b 1M
```

Step 6.2: EFI System Partition & Bootloader Generation

1. Format EFI system partition image (`efiboot.img`).
2. Populate with GRUB 2 EFI binaries (`B00TX64.EFI`) signed for Secure Boot compatibility.
3. Generate `grub.cfg` containing live boot parameters:

```
linux /boot/vmlinuz-linux-zen archisobasedir=doomos archisolabel=D00M0S_LIVE quiet splash co
initrd /boot/initramfs-linux-zen.img
```

Step 6.3: Mastering the Hybrid ISO with Xorriso

Generate the final bootable ISO image:

```
xorriso -as mkisofs \  
  -iso-level 3 \  
  -full-iso-9660-filenames \  
  -volid "DOOMOS_LIVE" \  
  -eltorito-boot isolinux/isolinux.bin \  
  -eltorito-catalog isolinux/boot.cat \  
  -no-emul-boot -boot-load-size 4 -boot-info-table \  
  -isohybrid-mbr isolinux/isohdpfx.bin \  
  -eltorito-alt-boot \  
  -e boot/grub/efiboot.img \  
  -no-emul-boot -isohybrid-gpt-basdat \  
  -output output/doomos-plasma-x86_64.iso \  
  iso_root/
```

Phase 7: Automated Testing & Deployment

Step 7.1: Virtual Machine Verification (QEMU)

Verify the newly created ISO boots to a full KDE Plasma desktop:

```
qemu-system-x86_64 \  
  -enable-kvm \  
  -m 4G \  
  -smp 4 \  
  -cpu host \  
  -bios /usr/share/ovmf/x64/OVMF.fd \  
  -cdrom output/doomos-plasma-x86_64.iso \  
  -vga virtio \  
  -device virtio-sound-pci,audiodev=audio0 \  
  -audiodev coreaudio,id=audio0
```

Step 7.2: Bare-Metal Deployment Checklist

1. Write the ISO to a USB drive:

```
sudo dd if=output/doomos-plasma-x86_64.iso of=/dev/sdX bs=4M status=progress oflag=sync
```

2. Boot test on target hardware:

- Verify Wi-Fi / Bluetooth scanning in SDDM.
 - Verify Wayland fractional scaling on 1080p, 1440p, and 4K displays.
 - Run Calamares installer and verify Btrfs subvolumes and snapshot recovery.
 - Launch Steam and verify GameMode and Xbox controller connectivity.
-

Complete Build Summary Table

Step	Action	Tools Used	Output Artifact
1	Build Scaffolding	Docker, Git	Build container environment
2	Rootfs Bootstrap	<code>pacstrap</code> , <code>mkinitcpio</code>	<code>rootfs/</code> + <code>vmlinuz-linux-zen</code>
3	Graphics & Desktop	Mesa, SDDM, KDE Plasma 6	Wayland desktop session
4	Subsystem Injection	Btrfs, Snapper, PipeWire	Auto-rollback & audio loopback
5	Installer Setup	Calamares, Python modules	Branded graphical installer
6	ISO Mastering	<code>mksquashfs</code> (<code>zstd</code>), <code>xorriso</code>	<code>doomos-plasma-x86_64.iso</code>
7	QA & Deployment	QEMU, OVMF, <code>dd</code>	Verified Golden Master OS

This playbook provides the end-to-end technical steps to construct, customize, and master DoomOS.